

6. EXPLICACIÓN DE TRIGGERS Y PROCEDIMIENTOS ALMACENADOS

La lógica de negocio de PIL ANDINA fue delegada directamente al servidor de base de datos mediante componentes programables encapsulados. Esto garantiza la integridad referencial, centraliza las reglas de negocio y reduce el tráfico de red, ya que la aplicación web en Flask solo debe invocar una instrucción simple en lugar de procesar complejas cadenas de código.

6.1. Procedimiento Almacenado de Despacho Mayorista (sp_despachar_pedido)

Este procedimiento fue programado para resolver el flujo crítico del Módulo 5 (Distribución) en una única transacción atómica e indivisible. Recibe como parámetro de entrada el identificador del pedido (p_id_pedido) y se encarga de cambiar su estado y descontar las existencias correspondientes.

- **La Importancia de la Lógica WHILE y Cursores:** A diferencia de una consulta estándar que opera sobre conjuntos estáticos, el despacho de un pedido mayorista exige procesar una estructura dinámica de N renglones (donde un pedido puede contener múltiples presentaciones de cerveza en diferentes cantidades). Para resolver esto de manera secuencial a nivel interno, el procedimiento utiliza un **Cursor** (cursor_pedido), el cual actúa como un puntero que extrae uno a uno los artículos solicitados desde la tabla detalle_pedidos.

La ejecución recurre a un **bucle de control WHILE** (o estructura LOOP controlada por una variable bandera de finalización v_finalizado), el cual itera de forma consecutiva fila por fila. En cada ciclo de la estructura de repetición, el procedimiento realiza las siguientes acciones de ingeniería:

1. Extrae el código de la presentación y la cantidad solicitada por el distribuidor.
2. Localiza el lote aprobado más antiguo con existencias disponibles en la bodega de la planta origen (cumpliendo con la regla de calidad FIFO).
3. Modifica directamente el stock actual en la tabla inventario_bodega.

4. Inserta de forma automática el registro de auditoría en la tabla movimientos_inventario bajo el concepto de '*Salida por Venta*'.

El bucle WHILE garantiza que no importa si el distribuidor solicitó 1 o 50 tipos diferentes de productos en la misma orden de compra, el motor los procesará todos de forma ordenada en microsegundos, rompiendo la iteración de manera segura únicamente cuando el cursor detecte que no quedan más filas por leer (NOT FOUND).

6.2. Trigger de Control Automatizado de Inventarios (tg_actualizar_stock_movimiento)

Un *Trigger* o disparador es un componente reactivo de seguridad que se ejecuta de forma automática ante un evento en la base de datos, sin intervención del usuario o de la aplicación Flask.

- **Lógica de Operación:** Este disparador está asociado de forma estricta a la tabla movimientos_inventario y se activa "**AFTER INSERT**" (inmediatamente después de que se registra cualquier transacción física de producto). Su función principal es mantener la sincronización absoluta en tiempo real de la tabla inventario_bodega mediante condicionales lógicos internos:
 - **Estructura IF (Entrada por Producción):** Si el campo tipo_movimiento es igual a '*Entrada por Produccion*', el trigger intercepta el valor y ejecuta un UPDATE sumando la cantidad ingresada al campo stock_actual del lote y bodega destino correspondientes.
 - **Estructura IF (Salida por Venta):** Si la transacción es una '*Salida por Venta*', el trigger aplica una operación aritmética de resta, disminuyendo de forma automática las existencias.
 - **Estructura IF (Traslado entre Bodegas):** Si se registra un traspaso logístico entre plantas, el trigger ejecuta de forma segura dos sub-operaciones atómicas en la misma celda de

tiempo: resta la cantidad en la bodega origen (id_bodega_origen) y la suma de manera inmediata en la bodega destino (id_bodega_destino).

Este diseño de disparadores encapsulados blindo la base de datos corporativa, asegurando que las existencias visibles en la aplicación web de PIL ANDINA sean siempre un reflejo exacto y verídico de la realidad física de los almacenes industriales.